

Handling Circular References

Cpt S 321 Homework Assignment

Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

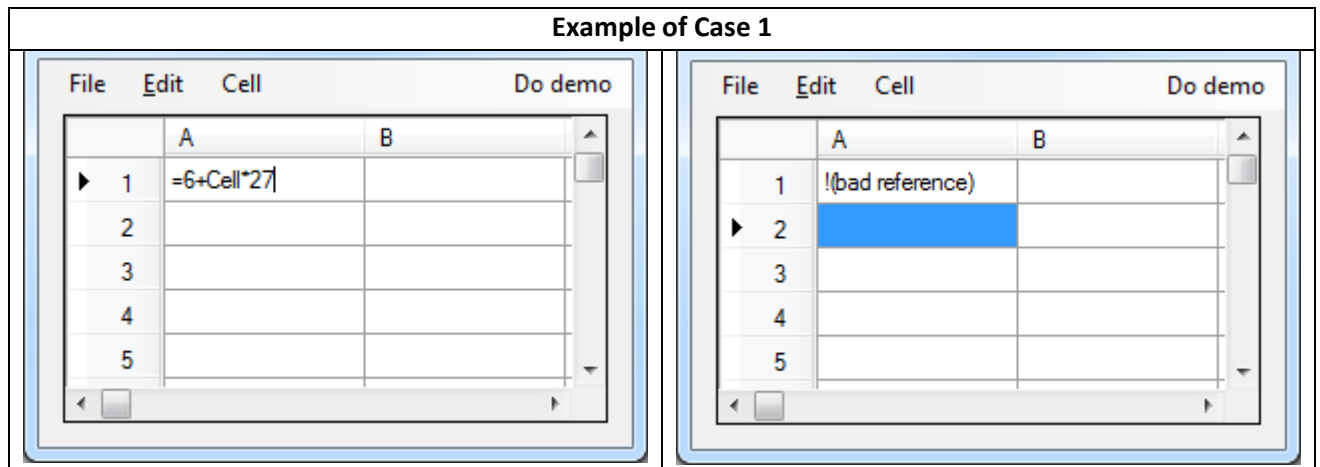
1. Create a branch called "Branch_HW10" and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. Work on the Assignment Instructions below.
4. Create a pdf document with your AI statement for the HW (call it "AI_HW10"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.
5. When you are done, merge the branch back to the master.
6. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW10".
7. Record a video to demo your HW (call it "Demo_HW10-FirstName_LastName").
8. On Canvas -> Assignments -> Upload your pre-recorded video by the HW deadline.
9. Don't forget to sign up and demo with the TAs.

Assignment Instructions:

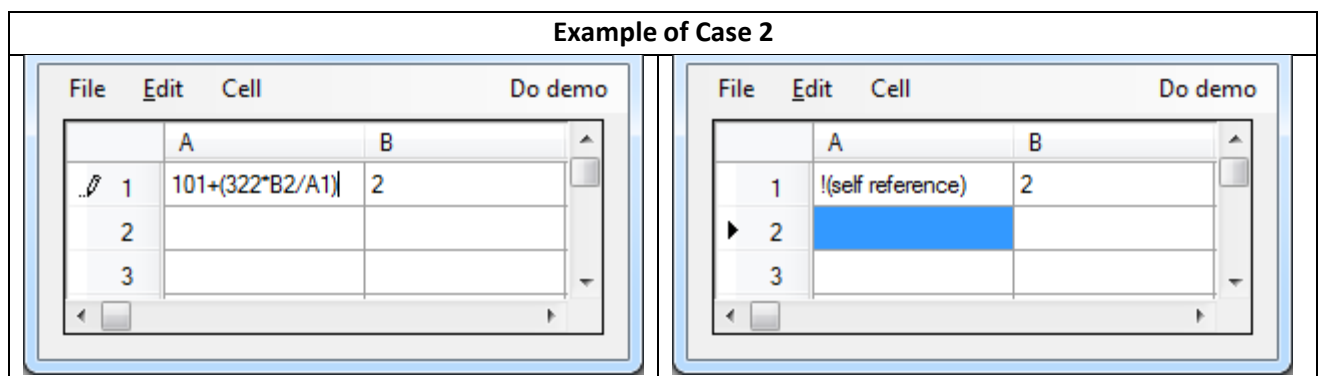
In this assignment you will give your spreadsheet application the ability to deal with problematic references in formulas. This may require some significant adjustments to your existing spreadsheet code BEFORE you add the new capabilities (i.e., perform some refactoring).

Below are the cases that you need to handle with respect to references in formulas. In real life, you'll need to come up with such lists on your own, but here you are given a list of possibilities to ensure that you don't miss anything. Handle all cases by setting some sort of simple descriptive message as the cell value (see video demo for examples).

1. The cell formula could reference something that doesn't exist on the spreadsheet. This could mean it's a cell name that's just beyond the range of what our spreadsheet supports, such as "Z12345". It could also just be a bad cell name, such as "Ba". Set the cell value to an error message as opposed to treating the non-existent cell's value as 0. Note that this is different than referencing a cell that DOES exist, but has no numerical value or no value at all, in which case you still treat its value as 0.



2. The cell formula could reference itself. Think of cell A1 and note that a self-referencing formula could be as simple as “=A1” or something more complex such as “=B2/(A1*A2)*7”. In ANY case where it contains itself in the formula the cell must set its value to an error string.



3. The cell formula could contain a circular reference. A simple example is that cell A1 has the formula “=B1” and cell B1 has the formula “=A1”. But note that the harder case is that the circular reference comes from more than one “step” in the evaluation chain. Consider a sheet with the following formulas:

	A	B
1	=B1*2	=B2*3
2	=A1*5	=A2*4

In this example no cell contains a reference to another cell that references directly back to it. But there is a circular reference chain nonetheless.

Handling the circular references:

You are only required to have one of the cells in the reference chain display the error message in the case of a circular reference. Or you could have all the problematic cells display an error message (or anywhere in between). That is up to you. But what you must ensure is that your app doesn't enter an infinite loop, cause a stack overflow, or (CRITICAL) fail to update properly when the formula is altered to eliminate the circular reference. This is by far the most difficult aspect of the assignment.

Final notes:

- Watch the video to see some simple examples of how your implementation of the application could respond to circular references.
- Remember that if a cell has a bad formula and displays an error message, resolving the error must trigger an update so that everything refreshes properly.

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	• Fulfill all the requirements above with no inaccuracies in the output and no crashes.
CR2 Quality of Version Control (1 point)	• Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). • Each commit should have cohesive functionality. • Commit messages should concisely describe what is being committed. • TDD should be used (i.e, write and commit tests first and then implement and commit functionality). • Include "TDD" in all commit messages with tests that are written before the functionality is implemented. • Use of a .gitignore. • Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	• No underscores in names of classes, attributes, and properties. • No numbers in names of classes or tests. • Identifiers should be descriptive. • Project names should make sense. • Class names and method names use PascalCasing. • Method arguments and local variables use camelCasing. • No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments	• Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>,

(1 point)	<returns>, <param>, and <exception> filled in as applicable. • All comment blocks use the format that is generated when typing “///” on the line above each entity. • There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	• Each file should only contain one public class. • Correct use of access modifiers. • Classes are cohesive. • Namespaces make sense. • Code is easy to follow. • StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. • Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	• Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. • Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). • <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	• Check the “Pre-recorded demos instructions” in the additional resources for details on the items below. • Version control history overview (< 30 sec.) • AI statement (< 30 sec.) • Design of the solution (2 min max) • Features showcase (2 min max)